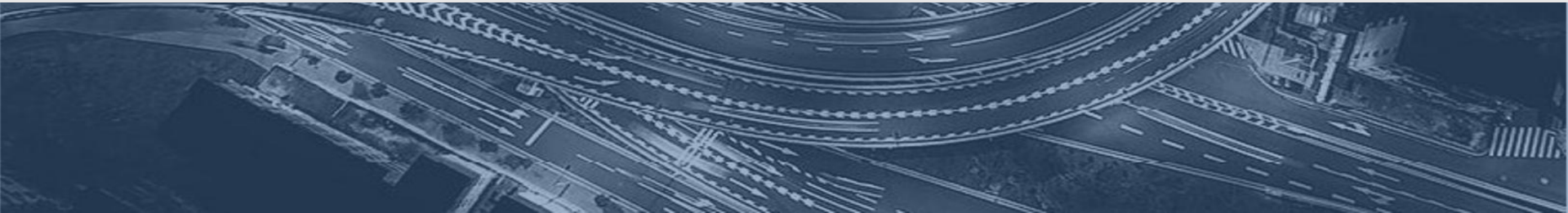
THE AUTOWARE FOUNDATION

Autoware Safety Island

National Taiwan University

國立臺灣大學

NEWS Lab 嵌入式系統暨無線網路實驗室



1. Why a Safety Island?

The main compute runs hundreds of nodes on Linux — complex, non-deterministic, hard to certify.

If it hangs, who stops the car?

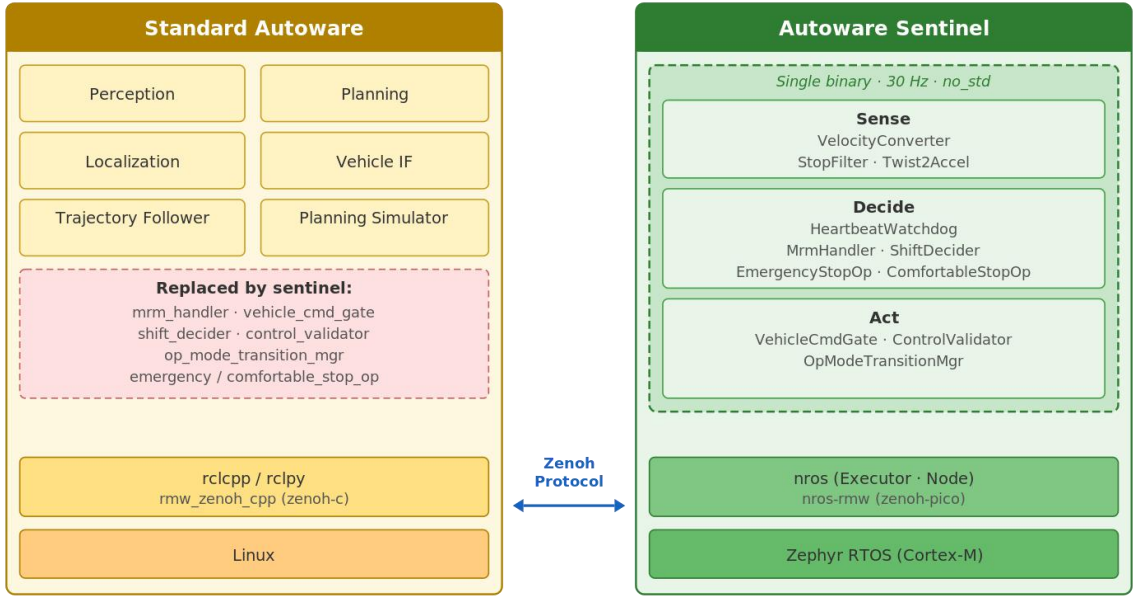
An independent MCU that monitors the main compute and brings the vehicle to a controlled stop.

	Main Compute	Safety Island
Hardware	Cortex-A (Orin, x86)	Cortex-M/R (MCU)
OS	Linux	RTOS / bare-metal
Complexity	Millions of LOC	Single binary
Certification	QM–ASIL B	ASIL C/D target
Failure mode	Detectable	Must not fail

Industry practice: NVIDIA DRIVE [1], Apex.AI [2], eSOL eMCOS [3] all use this pattern.

2. Autoware Sentinel — Our Reference Implementation

A single #![no_std] Rust binary replacing 7 Autoware safety nodes. 30 Hz deterministic loop. Zero heap allocation.



Heartbeat watchdog detects main compute failure (500 ms)

MRM handler activates emergency or comfortable stop

Vehicle command gate enforces rate/jerk limits

Control validator checks safety bounds

5 topics in, 30 topics out

Connected via Zenoh (rmw_zenoh)

3. nano-ros: The Foundation

A no_std ROS 2 runtime for embedded real-time systems. API compatible — same programming model across languages.

```rust
// Rust (rclrs-compatible)
let mut node = executor.create_node("safety")?;
let pub_ = node.create_publisher::<Control>("/cmd")?;
let _sub = node.create_subscription(
    "/vel", |msg: VelocityReport| { ... })?;
```

```c
// C (rclc-compatible)
nros_node_t node;
nros_publisher_t pub;
nros_create_node(&node, &exec, "safety");
nros_create_publisher(&pub, &node, "/cmd", ts);
```

Category	Support
Languages	Rust, C, freestanding C++14
Platforms	POSIX, Zephyr, FreeRTOS, NuttX, ThreadX, bare-metal
RMW	zenoh-pico, DDS-XRCE
Transport	TCP, UDP, serial (UART), IVC
Dev/test	Full stack runs in QEMU — no hardware needed

4. Why Rust for Safety-Critical Embedded?

no_std + alloc-free = bare-metal native. No runtime, no GC, no hidden allocations.

async/await with vendored runtime — the runtime is a library, not the language:

```
// RTIC: interrupt-driven tasks with async/await on Cortex-M
#[task(local = [executor], priority = 1)]
async fn net_poll(cx: net_poll::Context) {
    loop {
        cx.local.executor.spin_once(0);   // non-blocking I/O
        Mono::delay(10.millis()).await;    // yields to lower-priority work
    }
}

#[task(local = [publisher], priority = 2)]  // preempts net_poll
async fn publish(cx: publish::Context) {
    // This task runs at higher priority — guaranteed to meet deadline
    publisher.publish(&Control { velocity: 0.0 })?;
}
```

Rich ecosystem: 4,000+ no_std crates on crates.io. Community-maintained HAL traits, drivers, and board support packages.

5. The Bare-Metal Software Stack

"How can a program run without an OS?" On a microcontroller, your binary is the only software. No kernel, no scheduler, no main() that returns to a shell. The hardware boots straight into your code.

Application
Sentinel: watchdog, MRM, gate, validator

nano-ros
ROS 2 API: Node, Publisher, Subscriber, Executor

RTIC framework
Priority scheduler — maps tasks to HW interrupts

Hardware Abstraction Layer
Rust PAC/HAL: GPIO, UART, SPI, Ethernet, Timer

Cortex-M / Cortex-R hardware
NVIC, SysTick, MPU, DMA

Linux + ROS 2 vs. Bare-metal + nano-ros

Layer	Linux	Bare-metal
App	ROS nodes	Sentinel
Middleware	rclcpp/rclrs	nano-ros
Transport	rmw_zenoh	zenoh-pico (30k)
Scheduler	Linux kernel	RTIC (zero-cost)
Boot	U-Boot/GRUB	HAL + startup

Linux stack
~10M+ lines of code
Non-deterministic
QM–ASIL B

Bare-metal stack
~50k lines of code
Deterministic
ASIL C/D certifiable

Each layer is small and auditable. No virtual memory, no syscalls. Every function call is a direct branch.

5.2 How RTIC Replaces an RTOS

A traditional RTOS provides: (1) a scheduler, (2) synchronization primitives, (3) memory management. RTIC eliminates all three by leveraging hardware interrupt priorities directly.

Concept	Traditional RTOS	RTIC
Task	OS thread (dedicated stack)	Interrupt handler (shared stack)
Scheduling	Software scheduler (tick-driven)	Hardware NVIC (zero-latency)
Priority	Software priority queue	Hardware priority register
Mutex	Runtime lock (can deadlock)	Priority ceiling (deadlock-free)
Context switch	Kernel save/restore (~μs)	HW interrupt entry (~12 cycles)
Footprint	Kernel + per-thread stacks (KB)	No kernel, single stack (~1 KB)

Key insight: On Cortex-M, the NVIC (Nested Vectored Interrupt Controller) already implements a hardware priority-based preemptive scheduler. RTIC maps tasks onto interrupt vectors. The compiler statically verifies deadlock-freedom via the Stack Resource Policy (SRP) [5].

```rust
#[rtic::app(device = stm32f4xx_hal::pac,
            dispatchers = [SPI1, SPI2])]
mod app {
    #[shared]
    struct Shared {
        executor: Executor,        // nano-ros pub/sub
        watchdog_ok: bool,         // heartbeat status
    }

    #[init]
    fn init(cx: init::Context) -> (Shared, Local) {
        let dp = cx.device;        // raw peripherals
        let executor = Executor::open(&config).unwrap();
        control_loop::spawn().unwrap();
        (Shared { executor, watchdog_ok: true },
         Local { led, timer })
    }

    #[task(shared = [executor], priority = 1)]
    async fn net_poll(mut cx: net_poll::Context) {
        loop {
            cx.shared.executor.lock(|e| e.spin_once(0));
            Mono::delay(1.millis()).await;
        }
    }

    #[task(shared = [executor, watchdog_ok], priority = 2)]
    async fn control_loop(mut cx: ...) { /* 30 Hz */ }

    #[task(shared = [watchdog_ok], priority = 3)]
    async fn brake_check(mut cx: ...) { /* highest */ }
}
```

#[shared]

Resources accessed across tasks. Locked via priority ceiling — deadlock-free by construction.

#[init]

Runs once at boot. Configures hardware, creates executor, spawns tasks.

Priority 1: net_poll

Lowest priority. Handles Zenoh I/O in idle time. Preemptible by everything.

Priority 2: control_loop

30 Hz control cycle. Preempts net_poll, yields to brake_check.

Priority 3: brake_check

Highest priority. Never delayed by any other task. Checks heartbeat deadline.

5.4 Runtime Behavior & Benefits

```
Time ──────────────────────────────────────────────────────────────────────────▶

Priority 3  .........│....................│................│...
  (brake)    ↑ preempts             ↑ preempts
Priority 2  ...│└└└└└└└│└└└└└└└
```

6. Formal Verification: Miri, Kani, Verus

Three tools, three scopes — all run on the actual Rust source code.

Miri — runtime UB detector. Catches: use-after-free, uninitialized reads, unbounded loops.

Kani — bounded model checking (exhaustive)

```rust
#[kani::proof]
#[kani::unwind(301)]
fn convergence_from_20_mps() {
    let mut op = EmergencyStopOperator::new(
        Params::default());
    op.set_initial_velocity(20.0); // 72 km/h
    op.operate(true);

    let dt = 1.0 / 30.0;
    let mut step = 0u32;
    while op.velocity() > 0.0 && step < 300 {
        op.update(dt);
        step += 1;
    }
    assert!(op.velocity() == 0.0);
    // ← proved for ALL f32 paths
}
```

Verus — unbounded deductive proofs (zero cost)

```rust
proof fn ramp_phase_terminates(
    s: DecelState)
    requires
        s.acceleration_mms2
            > TARGET_ACCEL_MMS2,
    ensures ({
        let next = decel_step(s);
        next.acceleration_mms2
            < s.acceleration_mms2
            || next.acceleration_mms2
                == TARGET_ACCEL_MMS2,
    })

// Verus proofs verify end-to-end:
// reliability, latency bounds,
// state machine invariants
```


Proofs live alongside production code. Verus compiles away — zero runtime cost.

7. Network Safety & Target Platforms

TSN: Deterministic Network Guarantees

Standard	What it does	Safety value
802.1AS	Sub-microsecond time sync	Coordinated scheduling
802.1Qbv	Scheduled traffic windows	Provable worst-case latency
802.1Qci	Per-stream policing	Babbling idiot protection (FFI)
802.1CB	Frame replication	Seamless redundancy

ThreadX + NetX Duo [6] provides a complete TSN stack. Hardware: TSN-capable Ethernet MAC (NXP S32K3 [7], i.MX RT1180). IEEE 802.1DG-2025 defines the automotive TSN profile [8]. nano-ros adds E2E CRC-32 + sequence tracking (AUTOSAR E2E [9]).

Target Platforms

Platform	MCU	Transport	Status
QEMU MPS2-AN385	Cortex-M3	Ethernet / Serial	Working (dev/test)
Zephyr native_sim	x86_64	POSIX sockets	Working (CI)
NVIDIA Orin SPE	Cortex-R5F	IVC (shared memory)	Planned
NXP S32K3	Cortex-M7 lockstep	TSN Ethernet	Planned (ASIL D)

8. Discussion

Open questions for the working group:

Which Autoware nodes belong on the safety island?

What ASIL level should the safety island target?

Preferred hardware platform? (Orin SPE vs. NXP S32K3 vs. both)

TSN adoption timeline?

Integration with Autoware CI?

References:

[1] NVIDIA, "DRIVE OS 6.0 Safety," 2024

[2] Apex.AI, "Apex.OS — ISO 26262 ASIL D," AWF TSC 2021

[3] eSOL, "eMCOS POSIX — ASIL D RTOS"

[4] RTIC Framework — rtic.rs

[5] Baker, "Stack-Based Resource Allocation," IEEE RTSS 1990

[6] Eclipse ThreadX NetX Duo (MIT)

[7] NXP S32K3 — Cortex-M7 lockstep, ASIL D

[8] IEEE 802.1DG-2025 — Automotive TSN Profile

[9] AUTOSAR E2E Protocol Spec R22-11

Resources:

nano-ros: github.com/nicosio2/nano-ros

Safety island study: docs/research/autoware-safety-island-architecture.md

TSN assessment: docs/research/tsn-safety-island-assessment.md

ISO 26262 gap analysis: docs/research/autosar-iso26262-gap-analysis.md